

Procesamiento del Lenguaje Natural I

TEMA 3. REPRESENTACIÓN DEL TEXTO

Soto Montalvo Herranz

Objetivos

- Conocer diferentes modelos de representación de textos.
- Presentar la tipología de funciones de pesado.
- Conocer las técnicas de reducción de rasgos.

Índice

- Conceptos preliminares
- Modelos de representación
- Funciones de pesado
- Funciones de selección y reducción de rasgos
- Enlaces de interés
- Bibliografía

Índice

- **Conceptos preliminares**
- Modelos de representación
- Funciones de pesado
- Funciones de selección y reducción de rasgos
- Enlaces de interés
- Bibliografía

Conceptos preliminares

- Para procesar textos de forma automática es necesario extraer sus características o rasgos.
- El contenido textual se puede representar de diferentes formas (cadenas de caracteres, palabras, estructuras sintácticas, grafos de relaciones, etc.)
- La representación de un texto deberá ser fiel a su contenido, incluyendo la información necesaria para poder extraer el conocimiento útil que se espera obtener.
- La representación deberá ser adecuada a las especificaciones de los algoritmos que se empleen a continuación.

Conceptos preliminares

- Diferentes tareas de la minería de textos y diferentes tipos de textos pueden requerir diferentes representaciones.
- La representación de textos se refiere a cómo se estructuran y codifican los datos textuales para que las máquinas los puedan procesar y entender.
 - Las máquinas trabajan con números, no con palabras.

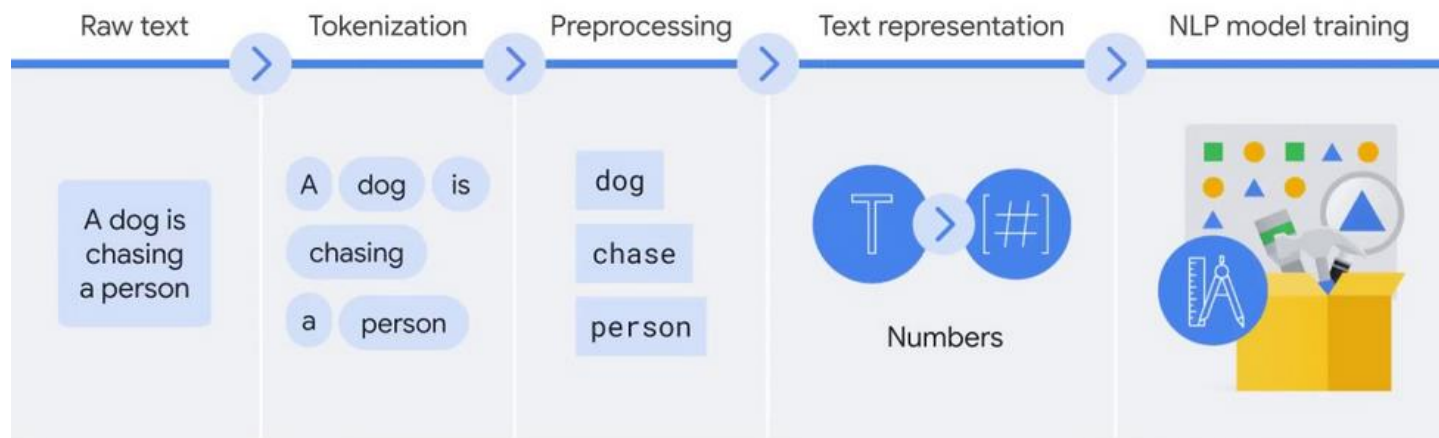


Imagen extraída de [2]

Conceptos preliminares

- Los modelos de representación más utilizados son:
 - Modelo de espacio vectorial (*Vector Space Model*, VSM)
 - Modelo probabilístico (*Probabilistic Topic Model*)
 - Modelo estadístico (*Statistical Language Model*)
 - Modelos semánticos vectoriales (*Vector Semantic Models*)
- En este curso nos centraremos en el modelo del espacio vectorial y en el semántico vectorial, dando nociones del resto de los modelos

Índice

- Conceptos preliminares
- **Modelos de representación**
- Funciones de pesado
- Funciones de selección y reducción de rasgos
- Enlaces de interés
- Bibliografía

Modelos de representación: vectorial

- Un modelo de representación vectorial de documentos se puede definir como una cuádrupla:

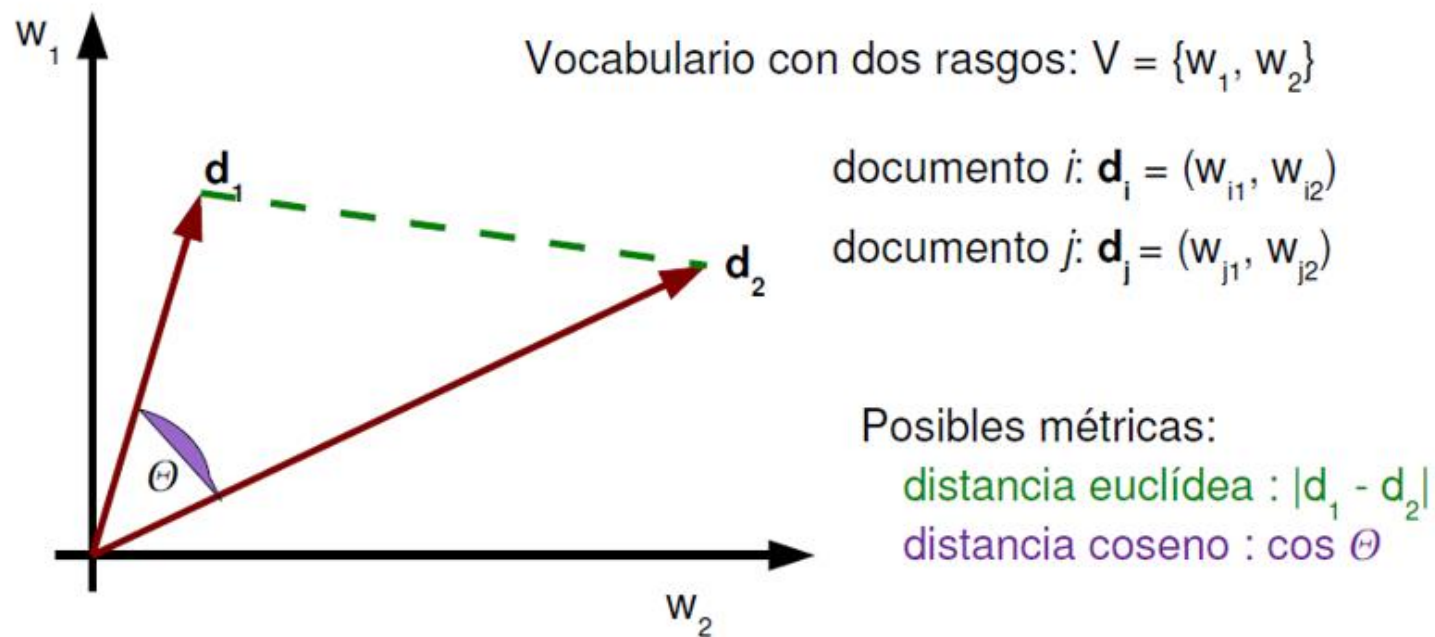
$$\langle X, B, \mu, F \rangle$$

- Conjunto de objetos **X** (cadenas) que constituye el **Vocabulario**
- **Álgebra B** que generaliza las relaciones entre los objetos
- Función de medida o **métrica, μ** , que establece la distancia (o similitud) entre objetos
- **Función de pesado F** que establece la proyección de cada objeto del espacio sobre la recta real

Modelos de representación: vectorial

- Los textos o documentos se modelan como conjuntos de rasgos (cadenas de caracteres: tokens, palabras, n-gramas, ...)
 - Se les puede asignar un peso
 - Todos forman el vocabulario de representación de los textos
- Los documentos pasan a ser representados como vectores dentro de un espacio euclídeo,
 - La distancia entre dos vectores estima su similitud como indicador de cercanía semántica
- Se pueden aplicar diferentes métricas μ para el cálculo de la distancia: distancia euclídea y coseno son las más populares, pero hay muchas otras

Modelos de representación: vectorial



Modelos de representación: vectorial

- **Modelo de espacio vectorial (VSM, *Vector Space Model*) [1]**

- Cada documento se representa por medio de un vector con dimensión T , donde T es el número de términos de la colección (número de términos del vocabulario).
- Cada término tiene un peso (asignado mediante una función de pesado F) que indica la relevancia de ese rasgo en el contenido del documento.
- Así, definir una representación dentro del VSM se reduce a encontrar una función de asignación de pesos $F: f(t_i) = t_{ij}$ para calcular el valor de cada componente del vector d_j .

Modelos de representación: vectorial

- **Modelo de espacio vectorial (VSM)**

- Dos representaciones de un mismo documento d_i (d_i y d'_i) a partir del mismo vocabulario serán diferentes siempre que el conjunto de valores t_{ij} que tomen las componentes de los vectores d_i y d'_i sean diferentes; es decir,

$$\vec{d}_j \neq \vec{d}'_j \text{ si y sólo si } \{i | t_{ij} \neq t'_{ij}, \forall i\} \neq \emptyset$$

Modelos de representación: vectorial

- **Modelo de espacio vectorial (VSM)**

- Asume el "principio de independencia" entre rasgos: aquellos que aparecen en un mismo texto no tienen relación entre sí y se pueden cuantificar individualmente.
- No tiene en cuenta el orden en el que aparecen los rasgos en el texto: la semántica de un documento queda reducida a la suma de los significados de los rasgos que contiene.
- ¿Las suposiciones anteriores son correctas?

Modelos de representación: vectorial

- **Modelo de espacio vectorial (VSM)**

- ***Bag of Words (BoW)***

- Es uno de los enfoques más básicos en la representación de textos.
 - Un documento se representa como una colección o “bolsa de palabras”, tantas como rasgos haya en el vocabulario.
 - En el vector resultante el valor de cada componente se calcula mediante una función de pesado.

- Matriz documentos-términos

	are	call	from	hello	home	how	me	money	now	tomorrow	win	you
0	1	0	0	1	0	1	0	0	0	0	0	1
1	0	0	1	0	1	0	0	1	0	0	2	0
2	0	1	0	0	0	0	1	0	1	0	0	0
3	0	1	0	1	0	0	0	0	0	1	0	1

Modelos de representación: vectorial

- **Modelo de espacio vectorial (VSM)**

- **Bag of Words (BoW)**

- Función de pesado más básica, binaria, valor 0 o 1 dependiendo de si el rasgo aparece o no en el documento (*One Hot Encoding*).

```
# One Hot Encoding
from sklearn.feature_extraction.text import CountVectorizer
```

```
# Sample corpus
```

```
corpus = ["Me gusta esta película",
          "Esta película es horrorosa",
          "La trama es confusa",
          "La trama es extraña"]
```

```
vectorizer = CountVectorizer(binary=True)
one_hot_encoded = vectorizer.fit_transform(corpus)
print(vectorizer.get_feature_names_out())
print(one_hot_encoded.toarray())
```

```
['confusa' 'es' 'esta' 'extraña' 'gusta' 'horrorosa' 'la' 'me' 'película'
 'trama']
[[0 0 1 0 1 0 0 1 1 0]
 [0 1 1 0 0 1 0 0 1 0]
 [1 1 0 0 0 0 1 0 0 1]
 [0 1 0 1 0 0 1 0 0 1]]
```


Modelos de representación: vectorial

- **Modelo de espacio vectorial (VSM)**
 - ***Bag of Words (BoW). One Hot Encoding***
 - Ventajas:
 - Se entiende bien.
 - Fácil de implementar.
 - Inconvenientes:
 - No hay posibilidad de relacionar unas palabras con otras.
 - Se generan matrices muy grandes y dispersas

Modelos de representación: vectorial

- **Modelo de espacio vectorial (VSM)**

- *Bag of Words (BoW).*

- Como función de pesado se mide la frecuencia de aparición de las palabras en el documento.

```
# One Hot Encoding
from sklearn.feature_extraction.text import CountVectorizer

# Sample corpus
corpus = ["Me gusta esta película, me ha parecido la mejor película del año",
          "Esta película es horrorosa",
          "La trama es confusa",
          "La trama es extraña"]

vectorizer = CountVectorizer()
bow_encoded = vectorizer.fit_transform(corpus)
print(vectorizer.get_feature_names_out())
print(bow_encoded.toarray())
```

	'año'	'confusa'	'del'	'es'	'esta'	'extraña'	'gusta'	'ha'	'horrorosa'	'la'	'me'	'mejor'	'parecido'	'película'	'trama'
[1 0 1 0 1 0 1 1 0 1 2 1 1 2 0]	1	0	1	0	1	0	1	1	0	1	2	1	1	2	0
[0 0 0 1 1 0 0 0 1 0 0 0 0 1 0]	0	0	0	1	1	0	0	0	1	0	0	0	0	1	0
[0 1 0 1 0 0 0 0 0 1 0 0 0 0 1]	0	1	0	1	0	0	0	0	0	1	0	0	0	0	1
[0 0 0 1 0 1 0 0 0 1 0 0 0 0 1]	0	0	0	1	0	1	0	0	0	1	0	0	0	0	1

Modelos de representación: vectorial

- **Modelo de espacio vectorial (VSM)**
 - ***Bag of Words (BoW).***
 - Ventajas:
 - Se entiende bien.
 - Fácil de implementar.
 - Captura algo de similitud semántica entre los documentos.
 - Inconvenientes:
 - No hay posibilidad de relacionar unas palabras con otras.
 - Se generan matrices muy grandes y dispersas

Modelos de representación: vectorial

- **Modelo de espacio vectorial (VSM)**
 - ***Bag of Words (BoW). TF-IDF.***

Term Frequency (TF):

TF = no. of times term occurrences in a document / total number of terms in a document

Inverse Document Frequency (IDF):

IDF = log base e (total number of documents / number of documents which are having term t)

TF-IDF Score:

*TF-IDF = TF * IDF*

Modelos de representación: vectorial

- **Modelo de espacio vectorial (VSM). Limitaciones:**

- **Alta dimensionalidad de la representación:** vectores muy largos y dispersos en colecciones extensas (contienen principalmente ceros) ya que muchas las palabras nunca ocurren en el contexto de otras.
- **Pérdida de correlación con las palabras (términos, rasgos) adyacentes:** algunas propuestas utilizan para representar un documento secuencias de símbolos (caracteres, palabras) denominadas n -gramas, donde n indica el tamaño de la secuencia.
- **Pérdida de posibles relaciones semánticas entre los términos de un documento:** algunas propuestas usan el Índice de Latencia Semántica (LSI) que es un método de análisis de coaparición entre rasgos y se basa en el análisis de latencia semántica (*Latent Semantic Analysis*, LSA), otras utilizan representaciones basadas en ontologías.

EJERCICIOS

(Ver primero el apartado de funciones de pesado)

Modelos de representación: probabilística

- **Modelo probabilístico de temas (*Probabilistic Topic Model*):**

- Los documentos se representan como una mezcla de temas, donde un tema es una distribución de probabilidad sobre palabras.
- Si un documento trata de un tema en particular, es de esperar que aparezcan palabras concretas en el documento con mayor o menor frecuencia: “carrocería”, “llantas” o “volante” aparecerán más a menudo en documentos sobre coches que en documentos de otros temas, mientras que palabras de uso general como “el”, “de” o “es” aparecerán igualmente en todos.
- Un documento típicamente trata múltiples temas en diferentes proporciones; por lo tanto, en un documento que es 80% sobre coches probablemente habría sobre ocho veces más palabras relacionadas con el tema “coches” que de otros temas.

Modelos de representación: probabilística

- **Modelo probabilístico de temas (*Probabilistic Topic Model*):**

- Los "temas" (topics) producidos por las técnicas de este modelado son grupos de palabras relacionadas (se van contando palabras y se mira cuáles coocurren).

Topic 1		Topic 2	
border	.4	space	.6
visa	.3	planet	.4
alien	.2	stars	.4
immigrant	.2	alien	.2

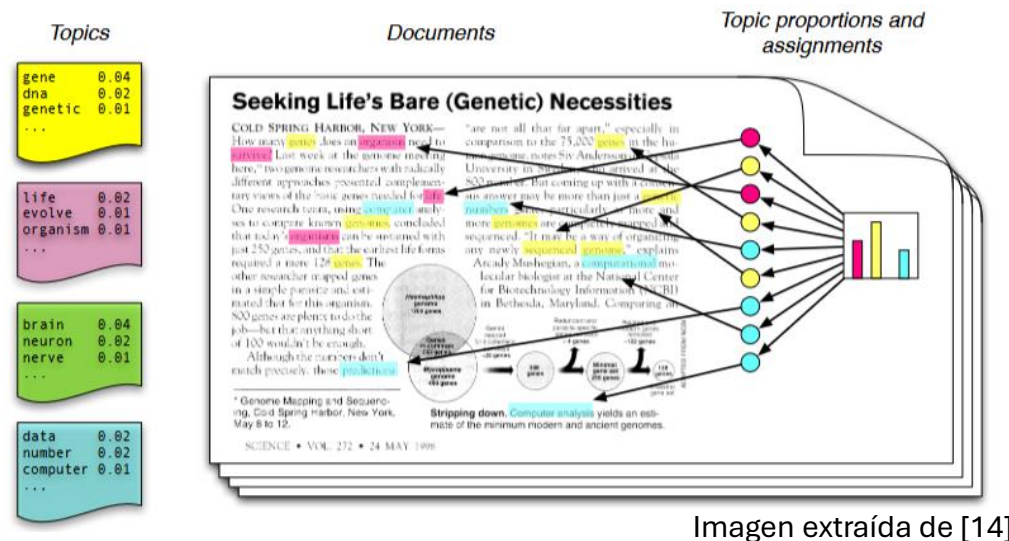
Document1: Topic 1: 1, Topic 2: 0
Document2: Topic 1: .1, Topic 2: .9
Document3: Topic 1: .85, Topic 2: .15
Document4: Topic 1: 0, Topic 2: 1

Imagen extraída de [15]

- 
- Dos posibles temas: inmigración y astronomía.
 - La probabilidad de que la palabra “borde” aparezca en el tema 1 es del 40%.
 - La probabilidad de que la palabra “espacio” aparezca en el tema 2 es del 60%.
 - El modelo, analizando la aparición de palabras en el documento 2, proyecta que en ese documento hay un 10% del tema 1 y un 90% del tema 2.

Modelos de representación: probabilística

- **Modelo probabilístico de temas (*Probabilistic Topic Model*):**



- Cada documento se modela como BoW.
- La mayoría de los enfoques comienzan generando la matriz documentos-términos.
- Documentos con palabras y frecuencias similares estarán más cerca en el espacio vectorial.
- Proximidad en el espacio vectorial => documentos que comparten temas.

- Existen distintos tipos de aproximaciones, una de las más comunes es la Asignación Latente de Dirichlet -- *Latent Dirichlet Allocation* (LDA).

Modelos de representación: probabilística

- **Modelo probabilístico de temas (*Probabilistic Topic Model*). LDA.**

Figure 2. Real inference with LDA. We fit a 100-topic LDA model to 17,000 articles from the journal *Science*. At left are the inferred topic proportions for the example article in Figure 1. At right are the top 15 most frequent words from the most frequent topics found in this article.

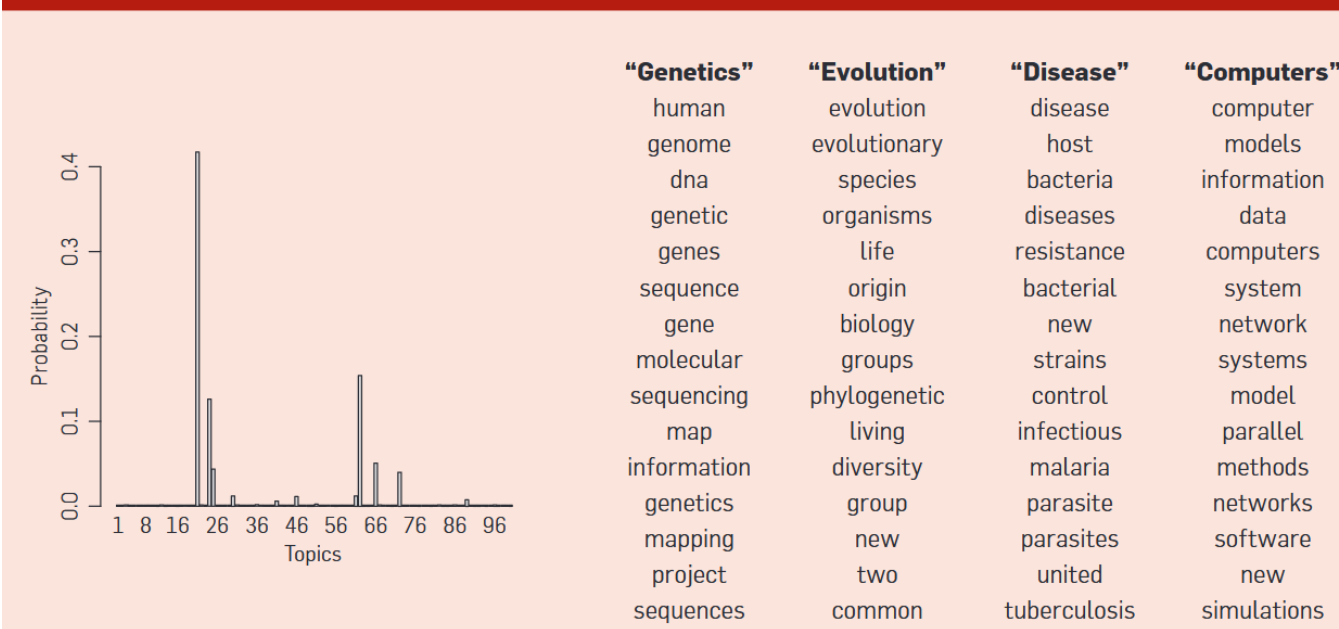


Imagen extraída de [14]

- Un tema o topic es una distribución de palabras sobre un vocabulario fijado.

Modelos de representación: estadística

- **Modelo de lenguaje estadístico de temas (Statistical Language Model):**
 - Es una distribución de probabilidad sobre secuencias de palabras o caracteres.
 - Modelo probabilístico que trata de predecir la siguiente palabra basándose en la secuencia de palabras que la preceden.
 - Uno de los modelos más utilizado es el modelo de n-gramas.
 - Se asume que la probabilidad de una palabra (o carácter) depende de las n palabras (o caracteres) anteriores.
 - Cuando en este modelo $n = 1$, es decir no hay dependencia de palabras previas, es equivalente a la bolsa de palabras (*Bag of Words*)

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales (*Vector Semantic Models*):**

- Se parte de la **hipótesis distribucional [3]**: las palabras que ocurren en contextos similares tienden a tener significados similares.
 - “*Usted conocerá una palabra por la compañía que posee*” [5]
- Relacionan la similitud en la distribución de las palabras y la similitud en su significado: palabras sinónimas tienden a aparecer en contextos similares.
- Se hace corresponder la diferencia de significado entre dos palabras con la diferencia en los contextos en los que aparecen.

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales (*Vector Semantic Models*):**

- En este modelo **a cada palabra se le asocia un vector** y se representa como un punto en algún espacio semántico multidimensional.



- Las palabras similares son “cercanas en el espacio semántico”.
- Modelo de espacio vectorial -> matriz de término-documento
- Modelo semántico vectorial -> matriz palabra-palabra (término-término), también denominada matriz término-contexto, porque las columnas son también palabras, no documentos.

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales (*Vector Semantic Models*):**
 - Matrices de co-ocurrencia

An automobile is a wheeled motor vehicle used for transporting passengers. A car is a form of transport, usually with four wheels and the capacity to carry around five passengers.

Transport for the London games is limited, with spectators strongly advised to avoid the use of cars.

The London 2012 soccer tournament began yesterday, with plenty of goals in the opening matches.

Giggs scored the first goal of the football tournament at Wembley, North London.

Bellamy was largely a Passenger in the football match, playing no part in either goal.

Imágenes extraídas de [6]

	wheel	transport	passenger	tournament	London	goal	match
automobile	1	1	1	0	0	0	0
car	1	2	1	0	1	0	0
soccer	0	0	0	1	1	1	1
football	0	0	1	1	1	2	1

automobile. car = 4

automobile. soccer = 0

automobile. football = 1

car. soccer = 1

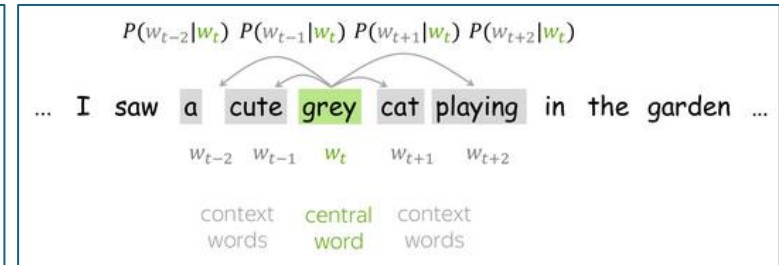
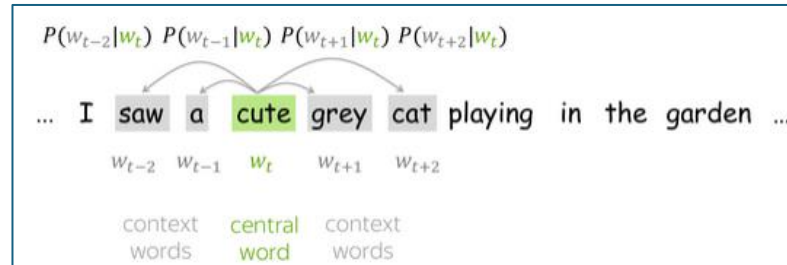
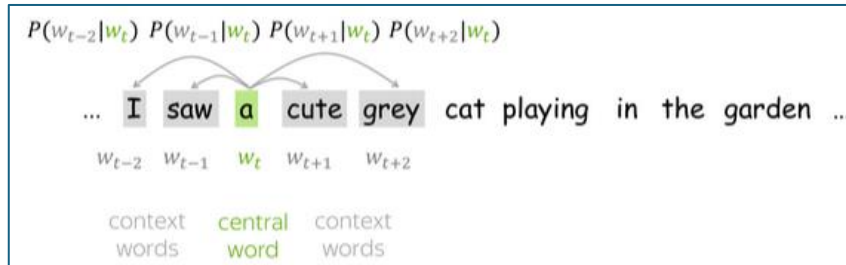
car. football = 2

soccer. Football = 5

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales (*Vector Semantic Models*):**

- ¿Cómo determinar el tamaño del contexto de la palabra objetivo? Se suele definir una ventana de varias palabras por delante y por detrás de una considerada.
- Se van recorriendo todas las palabras de los textos, y se va viendo qué palabras aparecen junto a ella. Así se obtiene un vector de probabilidades para cada palabra que contiene la probabilidad de que cada palabra del vocabulario aparezca en el contexto (entorno vecino) de la palabra objetivo.



Imágenes extraídas de [16]

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales (*Vector Semantic Models*):**

- Ejemplo de contexto: ventana de 4 palabras por delante y detrás de la palabra objetivo.

is traditionally followed by **cherry** pie, a traditional dessert
often mixed, such as **strawberry** rhubarb pie. Apple pie
computer peripherals and personal **digital** assistants. These devices usually
a computer. This includes **information** available on the internet

	aardvark	...	computer	data	result	pie	sugar	...
cherry	0	...	2	8	9	442	25	...
strawberry	0	...	0	0	1	60	19	...
digital	0	...	1670	1683	85	5	4	...
information	0	...	3325	3982	378	5	13	...

Figure 6.6 Co-occurrence vectors for four words in the Wikipedia corpus, showing six of the dimensions (hand-picked for pedagogical purposes). The vector for *digital* is outlined in red. Note that a real vector would have vastly more dimensions and thus be much sparser.

Imagen extraída de [4]

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales (*Vector Semantic Models*):**

- ¿Cómo medir la similitud entre dos palabras v y w ? Es necesario utilizar medidas de similitud entre los vectores. Ejemplo: medida del coseno.

$$\text{cosine}(\mathbf{v}, \mathbf{w}) = \frac{\mathbf{v} \cdot \mathbf{w}}{|\mathbf{v}| |\mathbf{w}|} = \frac{\sum_{i=1}^N v_i w_i}{\sqrt{\sum_{i=1}^N v_i^2} \sqrt{\sum_{i=1}^N w_i^2}}$$

- El valor del coseno puede ser:
 - 1 para vectores que apuntan en la misma dirección
 - 0 para vectores que son ortogonales
 - -1 para vectores que apuntan en direcciones opuestas

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales (*Vector Semantic Models*):**
 - Ejemplo: medida del coseno.

$$\text{cosine}(\mathbf{v}, \mathbf{w}) = \frac{\mathbf{v} \cdot \mathbf{w}}{|\mathbf{v}| |\mathbf{w}|} = \frac{\sum_{i=1}^N v_i w_i}{\sqrt{\sum_{i=1}^N v_i^2} \sqrt{\sum_{i=1}^N w_i^2}}$$

	pie	data	computer
cherry	442	8	2
digital	5	1683	1670
information	5	3982	3325

$$\cos(\text{cherry}, \text{information}) = \frac{442 * 5 + 8 * 3982 + 2 * 3325}{\sqrt{442^2 + 8^2 + 2^2} \sqrt{5^2 + 3982^2 + 3325^2}} = .018$$

$$\cos(\text{digital}, \text{information}) = \frac{5 * 5 + 1683 * 3982 + 1670 * 3325}{\sqrt{5^2 + 1683^2 + 1670^2} \sqrt{5^2 + 3982^2 + 3325^2}} = .996$$

Imagen extraída de [4]

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales (*Vector Semantic Models*):**

- Ejemplo: medida del coseno.

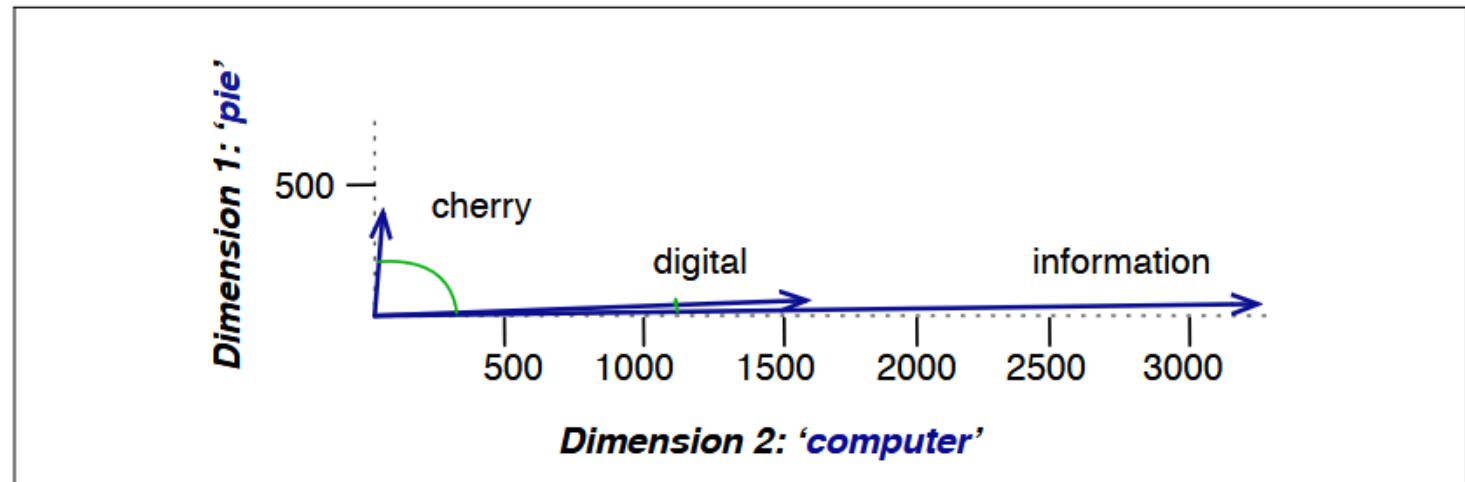


Figure 6.8 A (rough) graphical demonstration of cosine similarity, showing vectors for three words (*cherry*, *digital*, and *information*) in the two dimensional space defined by counts of the words *computer* and *pie* nearby. The figure doesn't show the cosine, but it highlights the angles; note that the angle between *digital* and *information* is smaller than the angle between *cherry* and *information*. When two vectors are more similar, the cosine is larger but the angle is smaller; the cosine has its maximum (1) when the angle between two vectors is smallest (0°); the cosine of all other angles is less than 1.

Imagen extraída de [4]

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales (*Vector Semantic Models*):**
 - Para evitar los vectores dispersos, se pueden limitar las dimensiones.
 - ¿Es la mejor opción utilizar la frecuencia como pesado en las matrices término-contexto?
 - Mejor TF-IDF, cuando las dimensiones son documentos (matriz documento-términos)
 - Mejor PPMI, cuando las dimensiones son palabras (matriz término-contexto)
 - Esta medida tiene sesgo para las palabras más infrecuentes, las palabras más raras tienen valores de PMI altos.

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales (*Vector Semantic Models*):**

- PPMI (*Positive Pointwise Mutual Information*), Información Mutua Puntual Positiva
- PMI (*Puntual Mutual Information*) mide la frecuencia con la que ocurren dos eventos x e y , comparados con lo que esperaríamos si fueran independientes.

$$\text{PMI}(w, c) = \log_2 \frac{P(w, c)}{P(w)P(c)}$$

- Puede dar como resultado valores negativos, por ello se utiliza PPMI, que en vez de valores negativos devuelve 0.

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales (*Vector Semantic Models*):**

- PPMI (*Positive Pointwise Mutual Information*), Información Mutua Puntual Positiva

$$\text{PPMI}(w, c) = \max(\log_2 \frac{P(w, c)}{P(w)P(c)}, 0)$$

- Si tenemos una matriz de co-ocurrencia F con W filas (palabras) y C columnas (contextos), donde f_{ij} es el número de veces que la palabra w_i aparece en el contexto c_j :

$$p_{ij} = \frac{f_{ij}}{\sum_{i=1}^W \sum_{j=1}^C f_{ij}}, \quad p_{i*} = \frac{\sum_{j=1}^C f_{ij}}{\sum_{i=1}^W \sum_{j=1}^C f_{ij}}, \quad p_{*j} = \frac{\sum_{i=1}^W f_{ij}}{\sum_{i=1}^W \sum_{j=1}^C f_{ij}} \quad \text{PPMI}_{ij} = \max(\log_2 \frac{p_{ij}}{p_{i*}p_{*j}}, 0)$$

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales (*Vector Semantic Models*):**

- ¿Qué cosas se pueden hacer con un modelo semántico vectorial? Ejemplos:
 - Encontrar las X palabras más similares a una palabra objetivo.
 - Similitud entre dos documentos por medio de su centroide (vector calculado a partir de los vectores de palabras):

$$d = \frac{\vec{w_1} + \dots + \vec{w_k}}{k}$$

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales (*Vector Semantic Models*):**

- La idea de la semántica vectorial es representar una palabra como un punto en algún espacio semántico multidimensional, las representaciones resultantes también reciben el nombre de **representaciones distribuidas**.
- Los vectores para representar palabras se llaman ***embeddings*** (embebidos) porque la palabra está embebida en un espacio vectorial en particular.

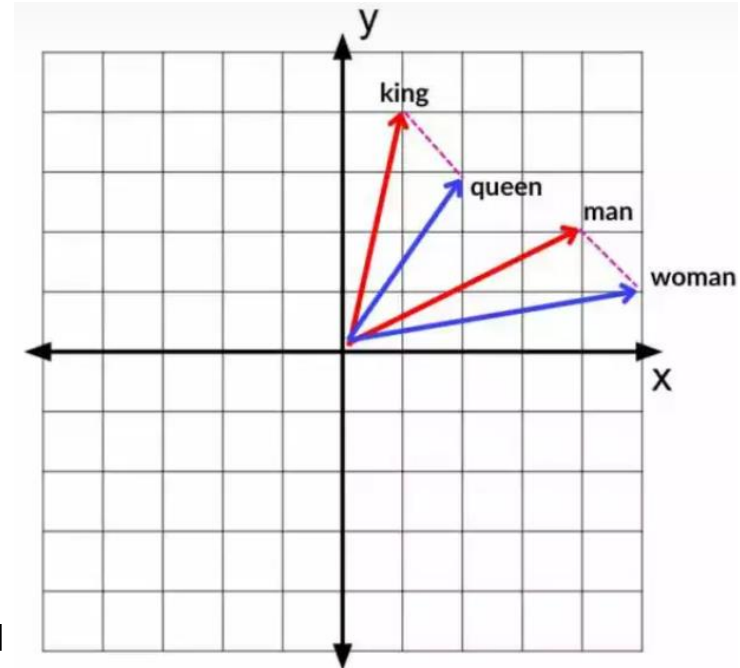


Imagen extraída de [7]

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales (*Vector Semantic Models*):**

- Embeddings

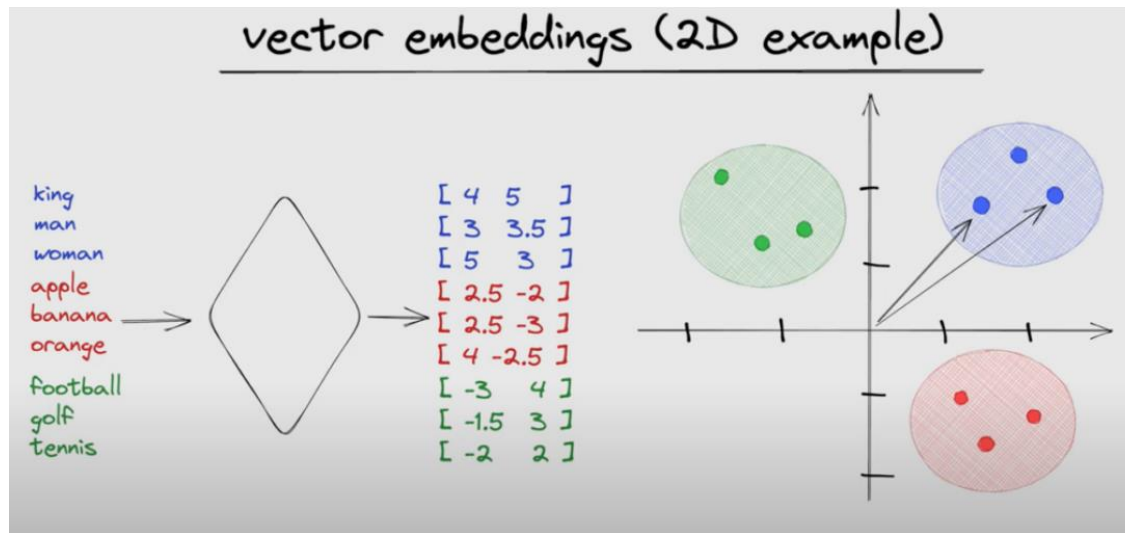


Imagen extraída de [8]

- Contexto: los embeddings se entrenan para capturar el contexto en el que aparecen las palabras.
 - “Banco” tendrá diferentes representaciones dependiendo del contexto (“institución financiera” o “asiento”).

Word embeddings: ejemplos de analogías

“España” - “Madrid” + “Lisboa” = Portugal Relación semántica

“Escribiendo” - “Hoy” + “Ayer” = Escrito Relaciones sintácticas

Imagen: AKTEK Ltd

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales (*Vector Semantic Models*):**

- Los modelos semánticos vectoriales son muy prácticos porque se pueden aprender automáticamente a partir de texto sin necesidad de etiquetado o supervisión.
 - Utilizan grandes corpus de texto para aprender las representaciones vectoriales.
- Un modelo de word embeddings está formado por un vocabulario (conjunto de palabras distintas de la colección de textos) y los embeddings (vectores).

Actualmente son la forma habitual de representación en muchas tareas de PLN:
traducción automática, análisis de opinion,
generación de textos, sistemas de recuperación de Información,
Chatbots, sistemas de recomendación, análisis de sentimientos,
etc.

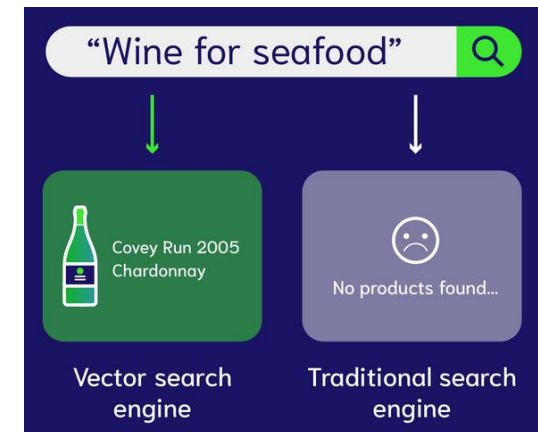


Imagen extraída de [9]

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales (*Vector Semantic Models*):**
 - Para trabajar con representaciones distribuidas una opción es generar nuestro propio modelo de vectores o bien se puede partir de un modelo ya entrenado.
 - Estas representaciones permiten utilizar vectores densos, en los que se limita el tamaño de las dimensiones y la mayoría de los valores de los vectores son distintos de cero.
 - Hay varios tipos o técnicas de word embedding disponibles:
 - Word2vec (desarrollado por Google)
 - Glove (desarrollado por Stanford)
 - FastTex (desarrollado por Facebook)

Modelos de representación: semánticos vectoriales

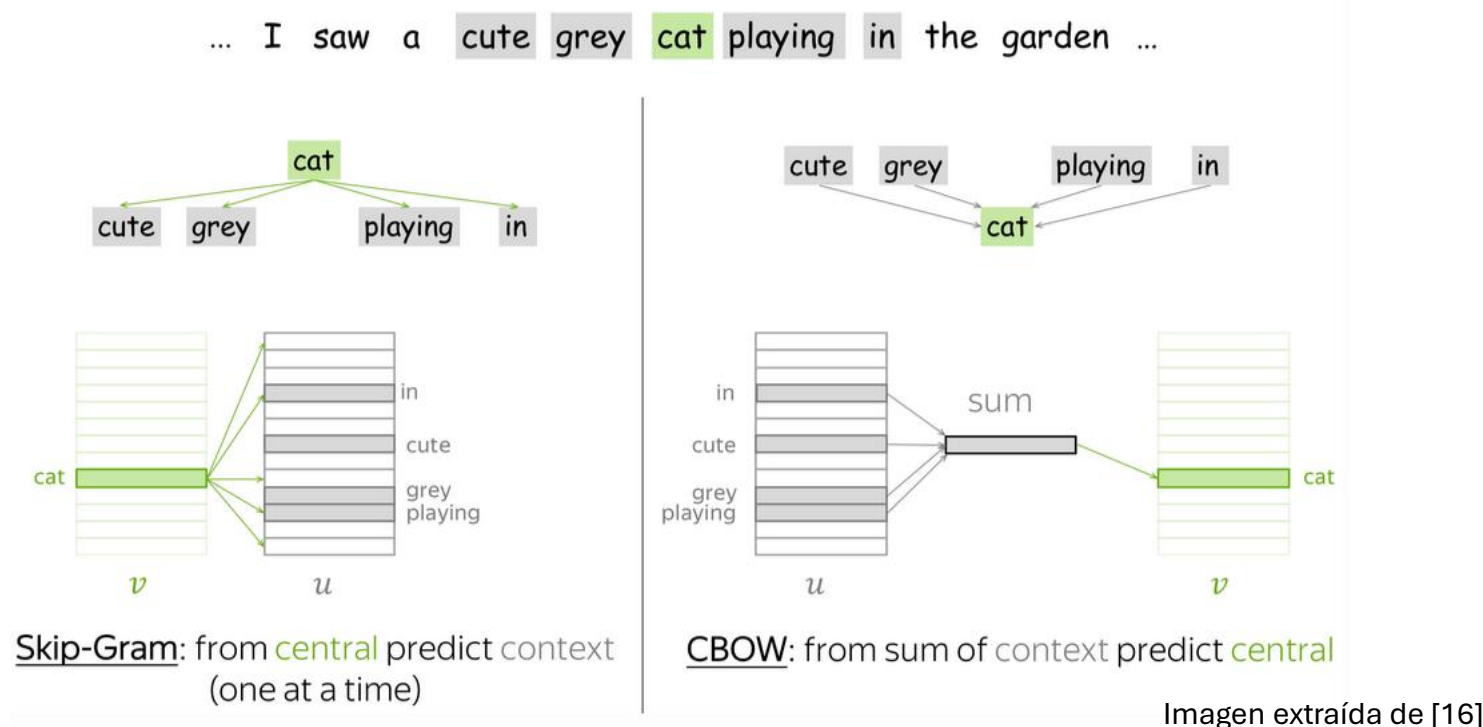
- **Modelos semánticos vectoriales. Word2Vec [10].**

- Word2Vec
 - Es una técnica que utiliza redes neuronales para aprender representaciones vectoriales de palabras.
 - Herramienta para entrenar/generar y usar word embeddings.
- Word2Vec utiliza modelos predictivos
 - Los modelos predictivos intentan predecir directamente una palabra a partir de sus vecinos
- Word2Vec utiliza una red neuronal de 2 capas que toma como entrada un corpus textual y genera los correspondientes word embeddings (que pueden ser la entrada de una red de aprendizaje profundo o servir para detectar relaciones entre palabras).

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales. Word2Vec [10].**
 - Existen dos arquitecturas o modelos neuronales principales:

- CBOW (Continuous Bag of Words): predice la palabra objetivo basada en el contexto (ventana de palabras). La entrada es una oración donde una de las palabras se ha ocultado (la que hay que predecir).
- Skip-Gram: predice el contexto más apropiado a partir de la palabra objetivo (que es la entrada).



Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales. Word2Vec [10].**

- Ambos enfoques son eficientes y permiten capturar relaciones semánticas y contextuales entre las palabras.
 - CBOW es más rápido que Skip-gram y obtiene mejores resultados para palabras muy comunes.
 - Skip-gram puede aprender buenas representaciones con conjuntos más pequeños que CBOW, y puede representar bien palabras raras.
- [Demo](#) para obtener embeddings, palabras similares, diferentes visualizaciones...
- [Demo](#) para probar modelos de embeddings de Word2Vec, ver las palabras más similares, la similitud entre dos palabras y analogía entre ternas de palabras.
- [Repositorio](#) con más de 200 modelos preentrenados sobre diferentes colecciones, dominios e idiomas.

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales. Word2Vec [10].**
 - Ejemplo de uso con la librería Gensim y CBOW.

Palabra más similar al contexto ['gato', 'negro']: blanco

```
import gensim
from gensim.models import Word2Vec

# Ejemplo de corpus de entrenamiento
sentences = [
    ['el', 'gato', 'negro'],
    ['el', 'perro', 'blanco'],
    ['el', 'gato', 'y', 'el', 'perro', 'son', 'amigos'],
    ['el', 'gato', 'negro', 'y', 'el', 'perro', 'blanco', 'juegan', 'juntos']
]

# Entrenamiento del modelo Word2Vec con CBOW
model = Word2Vec(sentences, vector_size=100, window=5, min_count=1)

# Guardar el modelo entrenado
model.save("word2vec_cbow.model")

# Cargar el modelo entrenado
model = Word2Vec.load("word2vec_cbow.model")

# Predecir la palabra más similar a un contexto dado
contexto = ['gato', 'negro']
prediccion = model.wv.most_similar(positive=contexto, topn=1)

print(f'Palabra más similar al contexto {contexto}: {prediccion[0][0]}')
```

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales. Word2Vec [10].**
 - Ejemplo de uso con la librería Gensim y Skip-Gram.

Contexto más probable para la palabra "gato":

amigos	(similitud: 0.1991206258535385)
blanco	(similitud: 0.17020152509212494)
juntos	(similitud: 0.1459505707025528)
negro	(similitud: 0.06408978253602982)
juegan	(similitud: -0.0027540279552340508)

```
# Ejemplo de corpus de entrenamiento
sentences = [
    ['el', 'gato', 'negro'],
    ['el', 'perro', 'blanco'],
    ['el', 'gato', 'y', 'el', 'perro', 'son', 'amigos'],
    ['el', 'gato', 'negro', 'y', 'el', 'perro', 'blanco', 'juegan', 'juntos']
]

# Entrenamiento del modelo Word2Vec con Skip-gram
model = Word2Vec(sentences, vector_size=100, window=2, min_count=1, sg=1)

# Guardar el modelo entrenado
model.save("word2vec_skipgram.model")

# Cargar el modelo entrenado
model = Word2Vec.load("word2vec_skipgram.model")

# Predecir el contexto más probable para una palabra dada
palabra = 'gato'
contexto = model.wv.most_similar(positive=[palabra], topn=5)

print(f'Contexto más probable para la palabra "{palabra}":')
for palabra, similitud in contexto:
    print(f'{palabra} (similitud: {similitud})')
```


Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales.**

Word2Vec [10].

- Modelos de la librería gensim

```
import gensim.downloader

# Show all available models in gensim-data
models_list = list(gensim.downloader.info()['models'].keys())

for model in models_list:
    print(model)
```

```
fasttext-wiki-news-subwords-300
conceptnet-numberbatch-17-06-300
word2vec-ruscorpora-300
word2vec-google-news-300
glove-wiki-gigaword-50
glove-wiki-gigaword-100
glove-wiki-gigaword-200
glove-wiki-gigaword-300
glove-twitter-25
glove-twitter-50
glove-twitter-100
glove-twitter-200
__testing_word2vec-matrix-synopsis
```

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales. GloVe [11].**
 - Word2Vec se enfoca en el contexto local.
 - GloVe utiliza una matriz de coocurrencia global que captura la probabilidad de que dos palabras aparezcan juntas en un corpus.

```
# Download the "glove-twitter-25" embeddings
glove_vectors = gensim.downloader.load('glove-twitter-25')

glove_vectors.most_similar('twitter')
```

```
[('facebook', 0.948005199432373),
 ('tweet', 0.9403423070907593),
 ('fb', 0.9342359900474548),
 ('instagram', 0.9104822874069214),
 ('chat', 0.8964963555335999),
 ('hashtag', 0.8885936141014099),
 ('tweets', 0.8878159523010254),
 ('tl', 0.8778461217880249),
 ('link', 0.8778210878372192),
 ('internet', 0.8753897547721863)]
```

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales. FastText [12].**

- FastText es una extensión del modelo Word2Vec en el que se consideran subpalabras o n-gramas.
 - No aprende un solo vector por cada palabra, sino que aprende vectores para todas las subpalabras que componen una palabra. Esto mejora la capacidad del modelo para manejar palabras desconocidas y variaciones morfológicas.
- El objetivo es obtener mejores representaciones para palabras poco frecuentes y poder generar vectores para palabras que no se encuentran en el vocabulario de los word embeddings enfocados en el contexto local.

Modelos de representación: semánticos vectoriales

- Modelos semánticos vectoriales. FastText [12].

```
Contexto más probable para la palabra "gato":  
amigos (similitud: 0.16541534662246704)  
blanco (similitud: 0.12396498024463654)  
el (similitud: 0.09178069233894348)  
juegan (similitud: 0.043360017240047455)  
son (similitud: 0.029763048514723778)
```

```
import gensim  
from gensim.models import FastText  
# Ejemplo de corpus de entrenamiento  
sentences = [  
    ['el', 'gato', 'negro'],  
    ['el', 'perro', 'blanco'],  
    ['el', 'gato', 'y', 'el', 'perro', 'son', 'amigos'],  
    ['el', 'gato', 'negro', 'y', 'el', 'perro', 'blanco', 'juegan', 'juntos']  
]  
  
# Entrenamiento del modelo FastText  
model = FastText(sentences, vector_size=100, window=5, min_count=1, sg=1)  
  
# Guardar el modelo entrenado  
model.save("fasttext_skipgram.model")  
  
# Cargar el modelo entrenado  
model = FastText.load("fasttext_skipgram.model")  
  
# Predecir el contexto más probable para una palabra dada  
palabra = 'gato'  
contexto = model.wv.most_similar(positive=[palabra], topn=5)  
  
print(f'Contexto más probable para la palabra "{palabra}":')  
for palabra, similitud in contexto:  
    print(f'{palabra} (similitud: {similitud})')
```

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales.**
 - **Comparativa (Word2Vec, Glove, FastText)**
 - Ventajas
 - Captura eficazmente las relaciones semánticas.
 - Eficaz para grandes conjuntos de datos.
 - Proporciona representaciones significativas de las palabras.
 - Inconvenientes
 - Puede tener problemas con palabras raras.
 - Ignora el orden de las palabras.

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales.**
 - **Comparativa (Word2Vec, **Glove**, FastText)**
 - Ventajas
 - Captura eficazmente las estadísticas globales del corpus.
 - Representa bien las relaciones semánticas y sintácticas.
 - Captura eficazmente las analogías entre palabras.
 - Inconvenientes
 - Requiere más memoria para almacenar las matrices de coocurrencia.
 - Menos eficaz con corpus muy pequeños.

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales.**
 - **Comparativa (Word2Vec, Glove, FastText)**
 - Ventajas
 - Mejor representación de palabras raras.
 - Capaz de manejar palabras fuera de vocabulario.
 - Representaciones de palabras más ricas gracias a la información sobre subpalabras.
 - Inconvenientes
 - Mayor tamaño del modelo debido a la información de n-gramas.
 - Mayor tiempo de entrenamiento en comparación con Word2Vec.

Modelos de representación: semánticos vectoriales

- **Limitación de los modelos de word embeddings**

- Se necesitan gran cantidad de textos (> millón de palabras) y tiempo para entrenar.
- No permiten representar sintagmas nominales o entidades nombradas (por ejemplo, “Estados Unidos”, “Gran Premio de Mónaco”).
- Hay palabras que aún siendo antónimos (“largo” y “corto”) tiene vectores próximos, porque suelen aparecer en contextos similares.
- Un único embedding por palabra no permite diferenciar palabras polisémicas (ej. “mono” puede ser animal, algo bonito o una prenda de vestir).

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales. Sense2Vec.**

- Sense2Vec [18] es una extensión de Word2Vec.
- Utiliza etiquetas de la categoría gramatical para distinguir los diferentes sentidos de una palabra.
 - Cada vector representa un sentido dentro de un contexto específico.
- Está entrenado sobre un corpus de comentarios de Reddit con anotaciones lingüísticas.

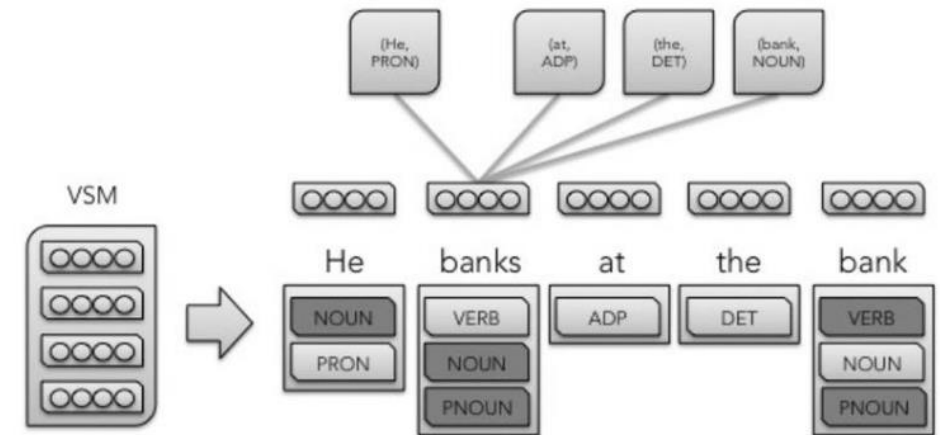


Imagen extraída de [17]

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales. Sense2Vec.**
 - Se puede usar de forma aislada cargando los vectores guardados previamente:

```
from sense2vec import Sense2Vec
s2v = Sense2Vec().from_disk("/path/to/reddit_vectors-1.1.0")
most_similar = s2v.most_similar("natural_language_processing|NOUN", n=10)
```

 **Important note:** To look up entries in the vectors table, the keys need to follow the scheme of `phrase_text|SENSE` (note the `_` instead of spaces and the `|` before the tag or label) – for example, `machine_learning|NOUN`. Also note that the underlying vector table is case-sensitive.

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales. Sense2Vec.**
 - Se puede integrar dentro del pipeline de spacy

```
import spacy
from sense2vec import Sense2VecComponent

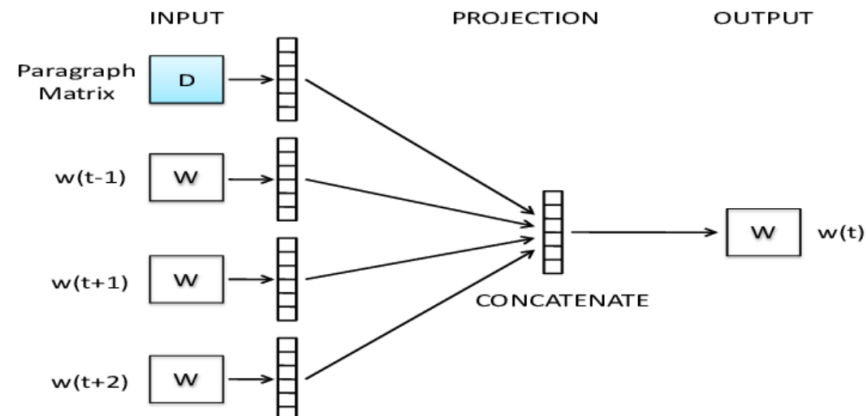
nlp = spacy.load("en_core_web_sm")
s2v = nlp.add_pipe("sense2vec")
s2v.from_disk("/path/to/s2v_reddit_2015_md")
```

```
doc = nlp("A sentence about natural language processing.")
assert doc[3:6].text == "natural language processing"
freq = doc[3:6]._.s2v_freq
vector = doc[3:6]._.s2v_vec
most_similar = doc[3:6]._.s2v_most_similar(3)
```

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales. Doc2Vec.**

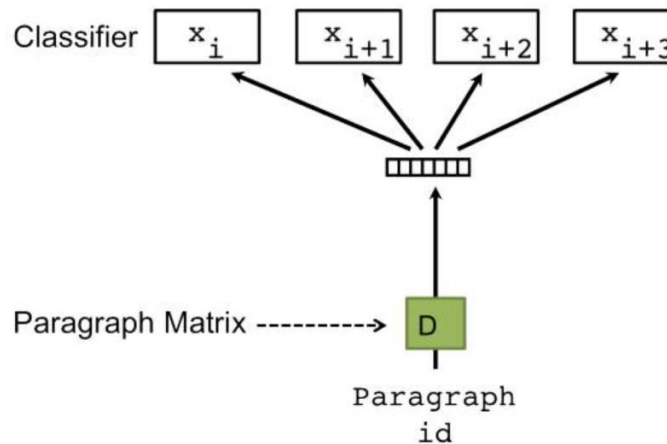
- Extensión de Word2Vec para generar embeddings de documentos.
- Dos variantes:
 - PV-DM (*Distributed Memory version of Paragraph Vector*)
 - Similar al modelo CBOW de Word2Vec.
 - Ahora se tienen en cuenta los vectores de palabras y un vector de documento para predecir la palabra objetivo.



Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales. Doc2Vec.**

- Extensión de Word2Vec para generar embeddings de documentos.
- Dos variantes:
 - PV-DBOW (*Distributed Bag of Words of Paragraph Vector*)
 - Similar al modelo Skip-gram de Word2Vec.
 - Ignora el orden de las palabras, siendo la fase de entrenamiento es más rápida.



Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales. Doc2Vec.**

- Comparativa PV-DM y PV-DBOW
 - PV-DM considera el orden de las palabras y el contexto del documento => más potente a la hora de calcular la semántica de los documentos.
 - PV-DBOW es más sencillo y rápido de entrenar. Es útil para ver la distribución de las palabras en un corpus.
- Ventajas de Doc2Vec
 - Genera embeddings de documento, que se pueden utilizar para clasificación y clustering de documentos, calcular la similitud entre ellos, etc.

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales. Word2Vec vs. Doc2Vec.**

- Si el corpus es pequeño o los documentos cortos, es posible que los vectores de documento no sean significativos.
 - Especialmente si la ventana supone una gran proporción del tamaño medio de los documentos => lo aprendido por los vectores de palabras será similar a lo aprendido por los vectores de documentos.
- Las palabras se entrenan con contextos más diversos => pueden tener un significado más generalizable
- La configuración del modelo influye claramente
 - Word2Vec se suele poner por defecto 5 iteraciones
 - Doc2Vec es mejor subirlo a 10 o 20 iteraciones
 - A veces Doc2Vec funciona mejor si se aumentan las dimensiones, pero hay que tener un compromiso con el coste computacional.

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales. Word2Vec vs. Doc2Vec.**

- Ejemplos de comparativas:

- Q. Chen, and M. Sokolova. “Specialists, Scientists, and Sentiments: Word2Vec and Doc2Vec in Analysis of Scientific and Medical Texts”. SN Comput Sci. 2021;2(5):414. ([Enlace](#))
 - I. R. Hendrawan, E. Utami and A. D. Hartanto, "Comparison of Word2vec and Doc2vec Methods for Text Classification of Product Reviews," 2022 6th International Conference on Information Technology, Information Systems and Electrical Engineering (ICITISEE), Yogyakarta, Indonesia, 2022, pp. 530-534. ([Enlace](#))
 - J.H. Lau, and T. Baldwin. “An Empirical Evaluation of doc2vec with Practical Insights into Document Embedding Generation”, Proceedings of the 1st Workshop on Representation Learning for NLP, pages 78–86, 2016. ([Enlace](#))

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales. Modelos contextualizados.**

- Las representaciones preentrenadas pueden ser independientes de contexto (“embeddings estáticos”) o contextualizadas, es decir, dependen de un contexto particular.
- Dentro de las representaciones contextualizadas, pueden ser unidireccionales o bidireccionales.
- Los modelos independientes del contexto como Word2Vec o GloVe generan una única representación vectorial para una palabra: “banco” de entidad financiera y “banco” de sentarse tienen la misma representación.
- Los modelos contextualizados generan una representación de cada palabra que está basada en las otras palabras **de la oración en la que aparece**: unidireccional si solo tiene en cuenta las previas y bidireccional si tiene en cuenta las previas y las siguientes.

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales. Modelos contextualizados.**

- Los primeros modelos contextualizados de uso Abierto fueron BERT (Google) y ELMo (Embeddings from Language Model), ambos para inglés.
- En la frase “*I accessed the bank account,*” representarán “*bank*” teniendo en cuenta “*I accessed the*” y “*account*” porque son bidireccionales, y en otra frase diferente “*bank*” podrá tener otra representación.



Imagen extraída de [13]

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales. Modelos contextualizados.**

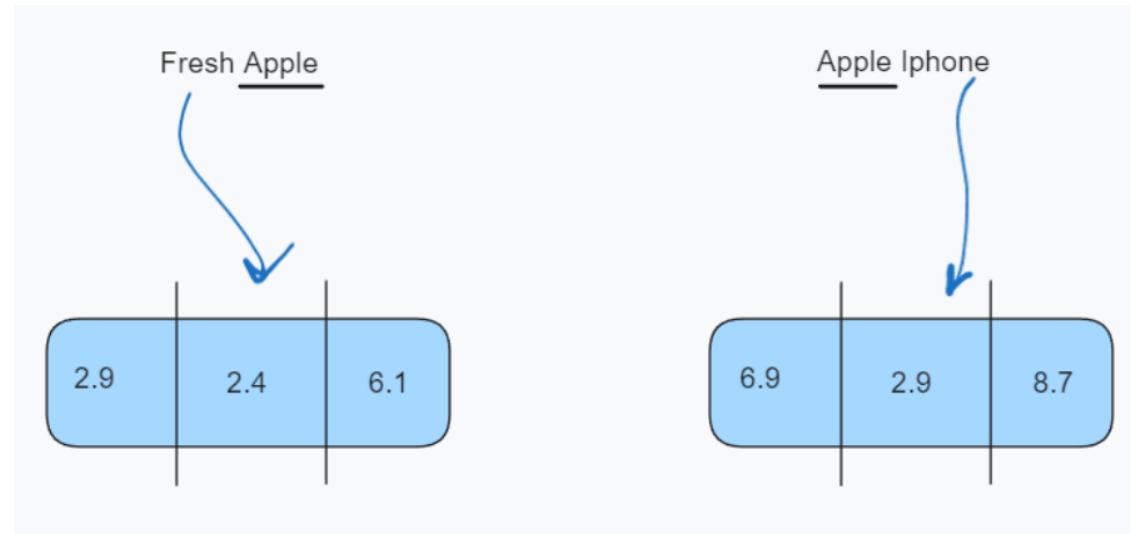
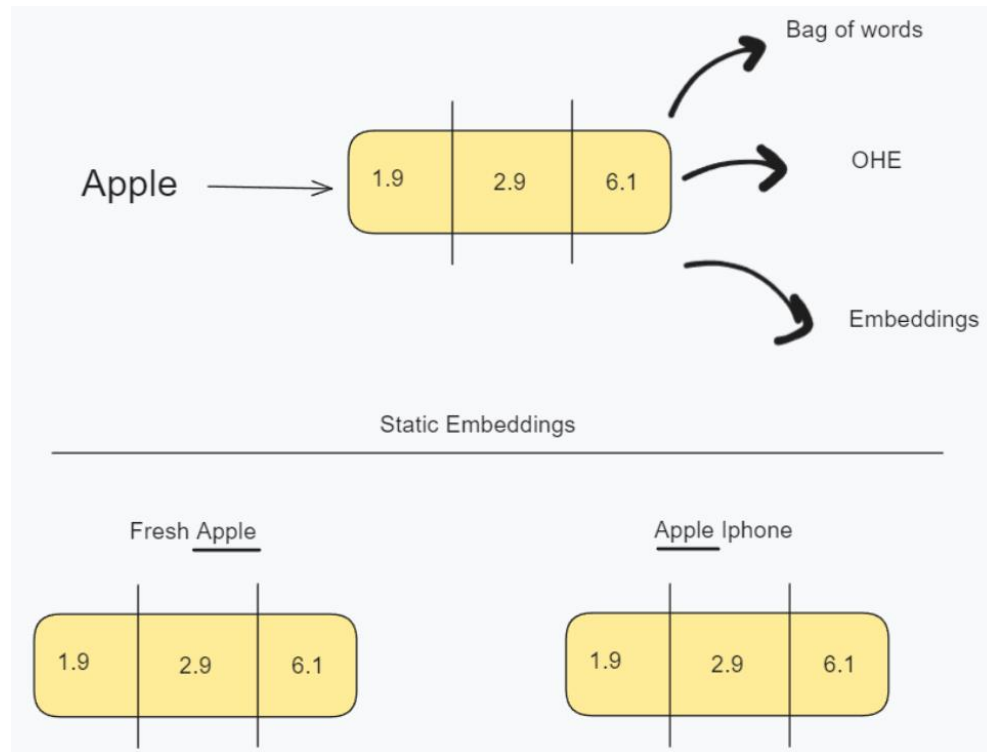


Imagen: Vinay Chaudhari LinkedIn

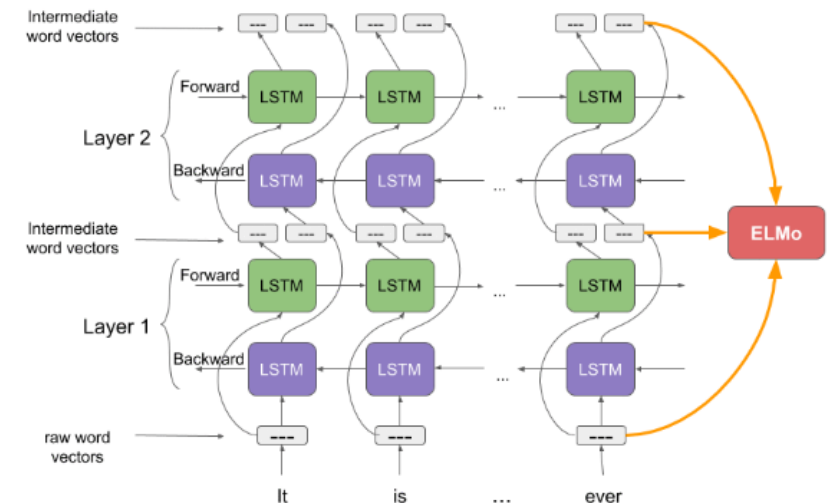
Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales. Modelos contextualizados. ELMo**

- ELMo permite representar una secuencia de palabras con su correspondiente secuencia de vectores, modela:
 - Características complejas del uso de las palabras (por ejemplo, sintaxis y semántica).
 - Cómo varían los usos en diferentes contextos lingüísticos (por ejemplo, polisemia). Representa el embedding de una palabra en una sentencia.

Se construye una arquitectura de aprendizaje profundo con dos capas en la que cada una de ellas contiene dos niveles de subcapas con múltiples redes neuronales recurrentes (RNN) con long-term short memory (LSTM).

Para obtener los embeddings del contexto se va leyendo el texto tanto de derecha a izquierda, como de izquierda a derecha, de forma lineal, utilizando lo que se denomina modelo bidireccional de lenguaje (BiLSTM).



Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales. Modelos contextualizados. BERT**

- BERT (Bidirectional Encoder Representations from Transformers) es un modelo de transformers preentrenado para la comprensión del lenguaje natural.
- Arquitectura de BERT: codificador de Transformers bidireccional de múltiples capas.
- El modelado de lenguaje que se lleva a cabo con BERT no es el de predecir la siguiente palabra dadas las palabras previas o posteriores, sino que enmascara una serie de palabras (un % de las palabras del texto original) e intenta predecir dichas palabras enmascaradas usando conjuntamente toda la información previa y posterior a las mismas.

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales. Modelos contextualizados. BERT**

- BERT puede ajustarse para tareas específicas mediante la adición de capas específicas para la tarea.
 - Como preentrenamiento (pre-training) extrayendo los embeddings de las palabras de una secuencia y utilizándolos para otras tareas posteriores, de ahí que se les llame modelos preentrenados.
- El fine-tuning permite a BERT realizar tareas como clasificación de texto, reconocimiento de entidades nombradas y respuesta a preguntas, etc.
- Existen modelos de lenguaje contextualizados para diferentes lenguas y dominios (mBERT que es multilingüe, BETO en español, RigoBERTa en español, etc.). Se verán en detalle más adelante.

Modelos de representación: semánticos vectoriales

- **Modelos semánticos vectoriales. Herramientas y bibliotecas.**
 - Los embeddings han revolucionado el PLN.
 - Algunas de las herramientas más populares:
 - TensorFlow, de código abierto, desarrollada por Google.
 - TensorFlow Embeddings: permite la implementación de embeddings.
 - TensorFlow Hub: contiene módulos preentrenados, incluyendo embeddings.
 - PyTorch, de código abierto, desarrollada por Facebook.
 - PyTorch embeddings: para crear embeddings de forma sencilla.
 - Gensim
 - Word2Vec y FastText
 - Es simple y eficiente.
 - Hugging Face Transformers, biblioteca de Transformers compatible con TensorFlow y PyTorch
 - Modelos preentrenados.
 - Interfaz intuitiva.

Índice

- Conceptos preliminares
- Modelos de representación
- **Funciones de pesado**
- Funciones de selección y reducción de rasgos
- Enlaces de interés
- Bibliografía

Índice

- Conceptos preliminares
- Modelos de representación
- **Funciones de pesado**
- Funciones de selección y reducción de rasgos
- Enlaces de interés
- Bibliografía

Funciones de pesado

- Las funciones de pesado (*term weighting functions*) constituyen funciones de proyección **F** dentro de la definición formal del modelo vectorial de representación de documentos.
- Dos tipos:
 - Funciones locales: solo utilizan información del propio documento para obtener el peso de un rasgo en el documento.
 - Funciones globales: utilizan también información de la colección para obtener el peso de un rasgo en el documento.

Funciones de pesado

- Algunas funciones locales:

- Binaria (*Binary*)

$$F : \text{Bin}(\vec{t}_i, \vec{d}_j) = \begin{cases} 1, & \text{si el rasgo } t_i \text{ aparece en } d_j \\ 0, & \text{si no aparece} \end{cases}$$

- Frecuencia (*Term Frequency*)

$$F : \text{TF}(\vec{t}_i, \vec{d}_j) = f_{ij}, \text{ frecuencia del rasgo } t_i \text{ en } d_j$$

Funciones de pesado

- Algunas funciones globales:
 - Frecuencia inversa del documento (IDF, *Inverse Document Frequency*)
 - Mide si se trata de un término común en la colección de documentos.

$$IDF(\vec{t}_i, \vec{d}_i) = \log\left(\frac{N}{1 + df(t_i)}\right)$$

donde $df(t_i)$ es el número de documentos de la colección en los que aparece el rasgo t_i y N el número de documentos total, se suma 1 al denominador ya que podría de otra forma ser 0 si el término no está en la colección.

Funciones de pesado

- Algunas funciones globales:
 - Frecuencia del término x Frecuencia inversa del documento (TF-IDF)

$$F : \text{TF-IDF}(\vec{t}_i, \vec{d}_j) = f_{ij} \times \log\left(\frac{N}{df(\vec{t}_i)}\right)$$

Un valor alto TF-IDF se alcanza con una elevada frecuencia del término en el documento dado y una pequeña frecuencia de ocurrencia del término en la colección completa de documentos

Índice

- Conceptos preliminares
- Modelos de representación
- Funciones de pesado
- **Funciones de selección y reducción de rasgos**
- Enlaces de interés
- Bibliografía

Funciones de selección y reducción de rasgos

- La **selección de rasgos** consiste en la transformación de un texto en el conjunto de rasgos que lo podrán representar.
- Estas transformaciones básicas se encuadran en lo que se denomina preprocesamiento de los textos.
- Tareas básicas de preprocesamiento son:
 - Tokenización
 - Normalización léxica
 - Truncado (Stemming)
 - Lematización
 - Eliminación de palabras vacías (*stopwords*)
 - Otros análisis lingüísticos básicos

Funciones de selección y reducción de rasgos

- La **reducción de rasgos** permite disminuir la dimensión de las representaciones que puede llegar a ser muy alta para colecciones de gran tamaño.
- Permite realizar una ponderación en base a la cual se pueden ordenar todos los rasgos de un vocabulario para seleccionar un subconjunto de ellos.
- La selección se puede hacer estableciendo un umbral de ponderación mínima o bien prefijando una dimensión reducida, generando así un vocabulario que resultará ser un subconjunto del vocabulario inicial.
- En muchas ocasiones las funciones de selección y reducción son funciones “orientadas a tarea”, es decir, que se definen en función de la tarea posterior.

Funciones de selección y reducción de rasgos

Las funciones presentadas en el apartado “funciones de pesado” pueden emplearse también como funciones de selección de rasgos:

- Con **funciones locales**, se seleccionan los rasgos con mayor peso **en cada documento** y se genera con ellos un vocabulario reducido.
- Con **funciones globales**, se seleccionan los rasgos con más peso dentro de **la colección** y se genera con ellos un vocabulario reducido.

Funciones de selección y reducción de rasgos

Selección de características/rasgos en aprendizaje automático:

- Mediante pruebas estadísticas se pueden seleccionar aquellas características que tienen la relación más fuerte con la variable de salida. Un ejemplo. En la librería [scikit-learn](#), en [Feature Selection](#) se describen diferentes estrategias de selección de rasgos.
- Se puede obtener la relevancia de cada característica del conjunto de datos utilizando la propiedad de importancia de la característica del modelo.
- La importancia de la característica da una puntuación para cada característica de los datos, cuanto mayor sea la puntuación, más importante o relevante será la característica para su variable de salida.
- La clase [feature importances](#) de `scikit-learn` permite extraer las X características más relevantes para un determinado clasificador.

Referencias

- [1] Salton, G., Wong, A., y Yang, C. A vector space model for automatic indexing. Communications of the ACM, 18(11):613–620. 1975.
- [2] Avelar, E. Feature Engineering in NLP. ([Enlace](#)).
- [3] Joos, M. Description of language design. JASA, 22, 701–708, 1950.
- [4] Jurafsky, D. y Martin, J.H. Speech and Language Processing. Draft of January 12, 2025. ([Enlace](#))
- [5] FIRTH, J.R. A synopsis of linguistic theory 1930-1955. Studies in linguistic analysis. 1957. pp.1–32.
- [6] CLARK, S. Vector Space Models of Lexical Meaning. In S. Lappin & C. Fox, eds. Handbook of Contemporary Semantics. 2014. pp. 1–43.
- [7] Van Otten, N. What is Text Representation. ([Enlace](#))
- [8] Vector Embeddings. ([Enlace](#))
- [9] Dascalescu, D. y Hasan, Z. Vector Embeddings Explained. 2023. ([Enlace](#))
- [10] Mikolov, T., Chen, K., Corrado, G. y Dean, J. Efficient Estimation of Word Representations in Vector Space, 2013. [arXiv:1301.3781](#)

Referencias

- [11] Pennington, J., Socher, R. y Manning, C.D. GloVe: Global Vectors for Word Representation. Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 1532–1543.
- [12] Bojanowski, P., Grave, E., Joulin, A. y Mikolov, T. Enriching Word Vectors with Subword Information. <https://arxiv.org/pdf/1607.04606.pdf>
- [13] Alammr, J. The Illustrated BERT, ELMo, and co. (How NLP Cracked Transfer Learning). ([Enlace](#))
- [14] Blei, D.M. Probabilistic Topic Models. Communications of the ACM, vol. 55(4), 2012. ([Enlace](#))
- [15] Murel, J. y Kavlakoglu, E. ¿Qué es el modelado de temas? 2024. ([Enlace](#))
- [16] Voita, L. NLP Course. ([Enlace](#))
- [17] Rosso, A. Word Embeddings y Semántica. ([Enlace](#))
- [18] Trask, A., Michalak, P. y Liu, J. sense2vec - A Fast and Accurate Method for Word Sense Disambiguation In Neural Word Embeddings, 2015. ([Enlace](#))